



Universidad
Indoamérica

UNITÉ 9 · FONDEMENTS DES SYSTÈMES D'EXPLOITATION

Le modèle propose, le système d'exploitation dispose

Universidad Tecnológica Indoamérica

OS natif pour l'IA

Du chat à l'agent

LLM

Texte → texte suivant.

Agent

Texte → **action** (outil) → résultat →
texte...

- Il agit au nom de l'utilisateur, mais **ce n'est pas l'utilisateur**.
- Il peut se tromper avec aplomb ou être piégé (injection de prompt).

Cinq responsabilités, trois plateformes

delete old_report.pdf

TD8 (ce cours)

utilisateur

agent (LLM)

appel d'outil

SE: décide

fichier / app

Identité de l'agent

socket UNIX 0600 + SO_PEERCRED (uid et pid du client)

Outils (connecteurs)

catalogue fermé de 11 outils ; analyseur strict

Consentement

agentd demande avant delete_file

Isolation

fork + exec d'agent_exec : setrlimit, alarm et Landlock

Audit

audit.log avec O_APPEND : une ligne par décision

Windows comme SE agentique

- **Agent workspace** avec un compte propre à l'agent (préversion)
- **Agent connectors** : serveurs MCP dans l'**On-Device Registry** (Insiders, déc. 2025)
- Principes : actions identifiables, autorisation de l'utilisateur, moindre privilège
- **Microsoft Execution Containers** (Build 2026) : processus · session · micro-VM

Sources et dates : manuel M9 §2.

Linux: des pièces à assembler

Landlock

chemins et réseau, sans
root

seccomp

quels appels

namespaces

· cgroups

ce qu'il voit · combien il
consomme

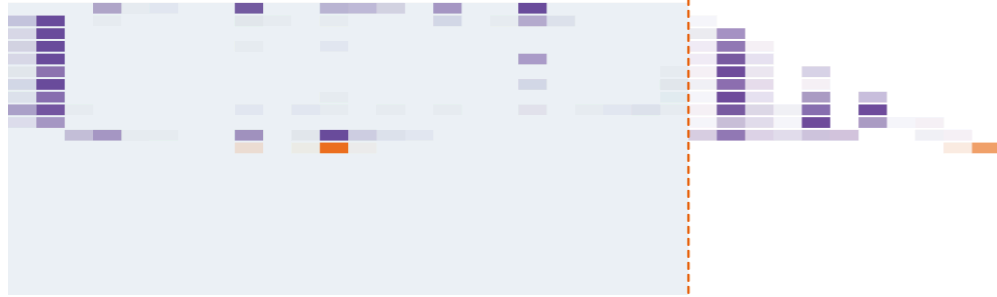
Distributions: Ubuntu annonce des *inference snaps* et des agents confinés (préversions dans la 26.10). Matériel: le sous-système `accel` pour les NPU.

Un LLM de l'intérieur: octets, cache KV, attention

requête et réponse, octet par octet

Q : `show me not a s . t x t` A : `read _ f`
`ile ( "`

carte d'attention: une ligne par étape, une colonne par octet



octet suivant: les 5 plus probables

n	100 %
w	0.0 %
y	0.0 %
p	0.0 %
-	0.0 %

```
read_file("I
```

174 656 paramètres en C, mesurés par le SE

Sortie réelle

```
$ ./tinylm -v "lista los archivos de docs"
""
list_files("docs")

model      : 174656 parameters, 3 layers, 4 heads, d=64, context 128 bytes
weights    : 698688 bytes mapped with mmap (MAP_PRIVATE, PROT_READ)
KV cache   : 201984 bytes per request (+ scratch)
tokens     : 33 of prompt (prefill) + 19 generated (decode)
time       : load 0.042 ms, inference 9.08 ms (5730 tokens/s)
page faults: 2 during load, 47 during the first inference (minor; major: 0)
```

- Les poids sont **projetés** (unité 6): le chargement prend quelques dixièmes de milliseconde et 2 défauts de page; les autres viennent quand on les utilise (unité 4)
- Le cache KV est une mémoire **par requête** ; les poids sont partagés par tous

Avoir raison 99 % du temps ne suffit pas

v1

99,4 % de correspondance exacte sur son test... mais au laboratoire, il a changé `viejo.txt` en `viefe.txt` : il avait vu peu de noms de ce genre.

v2

Le même modèle à 3 couches (174 656 paramètres), mais des données avec des noms plus variés et 12 000 étapes au lieu de 10 000 : 99,0 % sur 500 nouvelles requêtes, et il recopie correctement les noms rares.

1 % d'erreurs sur des milliers d'actions, ce sont des dizaines d'actions fausses : c'est pourquoi le SE **ne fait pas confiance** à la sortie du modèle (§5).

L'inférence, vue par le SE

PIÈCE	UNITÉ DU COURS
Poids avec <code>mmap</code> , partagés	U6
Défauts de page en touchant les poids	U4
Cache KV qui grandit à chaque token	U4
Un thread par requête dans un pool	U7
NPU / GPU comme périphériques	U1 · U3
L'agent parle à <code>agentd</code> par un socket local	U8

L'inférence, une charge pour l'OS

micro-LLM (TD8) · mesuré

code de tinyllm

tas : cache KV + zone de travail

52 / 128 positions · 78,0 KiB

mmap : fichier de poids (lecture seule)



171 pages de 4 Kio · 47 défauts à l'usage

pile

mémoire pour 8 agent(s)

poids partagés (mmap)



chaque agent copie les poids



■ poids ■ cache KV

une requête : 33 tokens en entrée (prefill) + 19 générés · 9,08 ms au total



≈ 175 µs par token sur un cœur

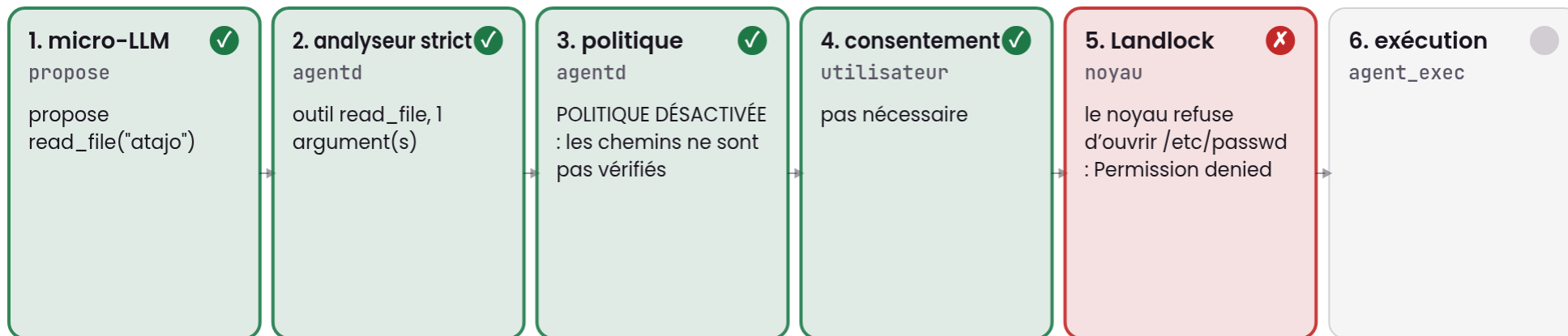
Le SE agentique du cours

Schéma

```
agent —(socket UNIX 0600)→ agentd — pool de 8 threads
    | SO_PEERCREDS : qui appelle (U8)
    | tinyllm (mmap des poids, cache KV)
    | analyseur strict → politique → consentement
    ▼
fork + exec agent_exec
    | setrlimit · alarm · no_new_privs
    | Landlock : seulement workspace/ et /proc (lecture)
    ▼
outil → résultat → audit.log
```

Défense en profondeur

requête :read atajo



ligne qu'agentd ajoute à audit.log:

```
2026-09-25T10:00:00 uid=1000 pid=4242 request="read atajo" call=read_file("atajo") decision=deny outcome=error (Permission denied)
```

TD8 + défi IA

Bash

```
# requêtes en anglais : le micro-LLM ne comprend que l'espagnol et l'anglais  
cd tds/bin/td8 && mkdir -p workspace && echo hello > workspace/notes.txt  
./agentd -w workspace &  
./agent "show me notes.txt"  
./agent "delete /etc/passwd"  
cat audit.log
```

Défi: ajoutez `copy_file` (données, réentraînement, politique, exécuteur) et concevez 5 attaques.



Universidad
Indoamérica

À RETENIR

Trois questions

1. Quelles cinq responsabilités un agent exige-t-il du SE?
2. Pourquoi la sortie du modèle est-elle un « texte non fiable »?
3. Quelle couche arrête le lien piégé si la politique échoue?